Tailscale for Personal Use: Private Mesh Networking Without the WireGuard Pain

> Tailscale is the easiest way to get a private network between all your devices — laptop, work computer, phone, VPS, home server, anything else — without dealing with port forwarding, dynamic DNS, or VPN server administration. Once it's set up, "SSH to my VPS" becomes "SSH to my VPS over a private network that nobody on the public internet can see."
>
> **NOTE. LAST VALIDATED: 2026-05.** Tailscale current (~1.78), Tailscale SSH GA, MagicDNS GA, Funnel GA. Free personal plan includes up to 100 devices and 3 users — far more than you'll need.

What you'll have at the end

- A **tailnet** — a private mesh network — connecting your devices (laptop, VPS, work computer, phone).
- Every device reachable by a short hostname (`apex`, `worklaptop`, `phone`) regardless of physical network or IP changes.
- **SSH to your VPS over the tailnet** — and then **closed port 22 on the public internet**, so the broader internet can't even attempt to brute-force SSH.
- **Internal services** (admin dashboards, Vaultwarden, Linkwarden, anything you self-host) reachable only by you and your devices, not the public.
- An **exit node** option — route all your laptop's traffic through your VPS as a VPN, for the times you're on hostile Wi-Fi.
- Optional: **Tailscale Funnel** for selectively exposing one specific service to the public internet through Tailscale's edge, no Caddy needed.

Prerequisites

- A VPS from vps-from-zero (we'll add Tailscale to it).
- Your laptop and any other devices you want on the tailnet.
- A GitHub, Google, Microsoft, or email account for Tailscale login (free; pick whichever).

Table of contents

1. What Tailscale actually is
2. Why for personal use
3. Install on the first device
4. Add the VPS
5. MagicDNS: hostnames that just work
6. Tailscale SSH: closing public port 22

7. Internal services: Caddy on the tailnet
8. Exit nodes: your VPS as a VPN
9. Subnet routes: bridging existing networks
10. ACLs: who can reach what
11. Funnel: selectively expose to the public
12. Removing a device
13. Troubleshooting
14. Alternatives considered
15. Quick reference

---

1. What Tailscale actually is

Tailscale is a **mesh VPN built on WireGuard** with a control plane that handles all the painful parts.

Mechanically:

- Every device runs the Tailscale client (Linux daemon, macOS/Windows app, iOS/Android app).
- The client talks to **Tailscale's coordination server** to register itself and learn about other devices in your "tailnet" (your personal network).
- For each peer, the client gets a public WireGuard key and an endpoint. It then establishes a **direct peer-to-peer WireGuard tunnel** to that peer.
- Tailscale handles **NAT traversal** (so devices behind home routers, mobile networks, etc. can still reach each other directly) using a mix of STUN, hole-punching, and fallback relays (DERP servers).
- **All actual traffic between your devices is end-to-end encrypted WireGuard.** Tailscale's control plane doesn't see your traffic — only the metadata needed to coordinate connections.

The "magic" is everything around the WireGuard part: device authentication, key rotation, peer discovery, NAT traversal, DNS. WireGuard alone would require you to manually exchange keys, manually configure peers, manually maintain endpoint addresses. Tailscale automates it.

The tailnet

Your tailnet is a private IPv4 (`100.x.x.x` from the CGNAT range) and IPv6 (`fd7a:115c:a1e0::/48`) address space that only your devices can reach. Each device gets a stable IP within the tailnet — it doesn't change as the device moves between networks.

Hostnames are auto-assigned (and overridable) — `apex.tail-scale.ts.net` is your VPS, `worklaptop.tail-scale.ts.net` is your work computer, etc. With MagicDNS (§5), bare hostnames work too: `ssh apex` just works.

---

2. Why for personal use

Compared to what you'd do without Tailscale:

Goal	Without Tailscale	With Tailscale
Reach your VPS from anywhere	Open port 22 (or other) on the public internet, accept brute-force attempts	Tailscale connection; port 22 stays closed publicly
Access internal services on the VPS	Public DNS + Caddy + Cloudflare proxy + basic auth	Tailnet-only access; nothing public
Connect laptop to home network	Set up dynamic DNS + port forwarding + WireGuard server + manual peer config	Install Tailscale, sign in, done
Tunnel laptop traffic through VPS (VPN)	OpenVPN server, profile management, port forwarding, ACLs	Enable "exit node" on the VPS; pick it from the laptop UI
Connect another person's device temporarily	Generate WireGuard keys, transfer config, hope NAT cooperates	Send them an "invite to tailnet" link, they accept
Survive an IP address change on a peer	Manually update peer config, restart WireGuard, hope DNS catches up	Tailscale auto-discovers the new endpoint

The pattern: **all the operational pain of running your own VPN goes away**, while you keep WireGuard's performance and security guarantees.

Free for personal use (up to 100 devices, 3 users). At the scale you're working at, you'll never approach those limits.

> **IMPORTANT. TAILSCALE'S COORDINATION SERVER IS A TRUST DEPENDENCY.** If they go down, existing tunnels keep working but you can't add new devices or update keys. If they're compromised, an attacker could in theory inject themselves into your tailnet (though they still can't read existing peer-to-peer traffic, since WireGuard keys are generated client-side). For higher-paranoia setups, run Headscale (open-source compatible coordination server) on your own infrastructure. For personal use, trust Tailscale Inc. — they've earned it.

---

3. Install on the first device

Start with the device you're physically at right now — usually your laptop. This establishes your tailnet.

Linux (WSL Ubuntu or native)

```
curl -fsSL https://tailscale.com/install.sh | sh
sudo tailscale up
```

It prints a URL. Open it in a browser, sign in with GitHub/Google/Microsoft/email. After approval, the device joins your new tailnet.

macOS

Install from the App Store or `brew install --cask tailscale`. Click the menu bar icon → "Log in."

Windows

Install from tailscale.com/download. Sign in via the system tray app.

iOS / Android

Install from the App Store / Play Store. Sign in.

Verify

After signing in:

```
tailscale status
# 100.x.x.x  worklaptop  joshuawjulian@  linux   -
```

That's it. Your first device is on the tailnet. The IP `100.x.x.x` is your device's tailnet address.

> **NOTE. THE FIRST DEVICE YOU SIGN IN WITH IMPLICITLY CREATES YOUR TAILNET** under your chosen identity provider's account. There's no separate "create tailnet" step. Subsequent devices that sign in with the same account join the same tailnet.

---

4. Add the VPS

On your VPS:

```
ssh myvps
curl -fsSL https://tailscale.com/install.sh | sudo sh
sudo tailscale up --ssh
```

It prints a URL. Open it on your laptop, click approve.

Two important flags:

- `--ssh` — enables Tailscale SSH, which lets you SSH to this VPS over the tailnet without using OpenSSH host keys (and lets you delete the open public SSH port later).
- For a server you want to be **stable** (not require periodic re-auth): `--auth-key=<key>` with a long-lived auth key from the admin console. For the initial setup, the browser flow is simpler.

Verify on your laptop:

```
tailscale status
# 100.x.x.x  worklaptop  joshuawjulian@  linux   -
# 100.y.y.y  apex        joshuawjulian@  linux   -
```

Both devices listed. They can now reach each other over tailnet IPs:

```
# From laptop
ping 100.y.y.y      # works, tunneled
ssh julian@100.y.y.y    # works, tunneled
```

> **TIP. GIVE YOUR VPS A MEANINGFUL HOSTNAME** in the Tailscale admin console (Machines tab → click the device → "Edit machine name"). Default is the OS hostname; "ubuntu-22-04-x64" is unhelpful. Rename to *apex* or *vps* so it's memorable.

---

5. MagicDNS: hostnames that just work

MagicDNS makes every device on your tailnet reachable by its short hostname. Instead of `ssh julian@100.y.y.y`, you can `ssh julian@apex`.

Enable MagicDNS

Tailscale admin console → DNS → toggle "MagicDNS." It uses Tailscale's own DNS server for your tailnet zone (`*.tail-scale.ts.net`, but the short names work too).

Now you can:

```
ping apex                          # works
ssh julian@apex                    # works (uses tailnet IP)
ssh julian@apex.tail-scale.ts.net  # also works (full FQDN)
curl http://worklaptop:8080        # if there's a dev server running on the work laptop
```

> **TIP.** `TAIL-SCALE.TS.NET` is a placeholder — your actual tailnet's domain is generated when you create it (something like *`tail-c0ffee.ts.net`*). The short hostname works the same regardless.

Custom DNS records

In the admin console under DNS → "Search Domains" you can add additional DNS resolution rules. Useful when you have internal services you want to reach by friendly names that aren't just hostnames.

---

6. Tailscale SSH: closing public port 22

This is where Tailscale really pays for itself. With `--ssh` enabled on the VPS, Tailscale provides SSH access over the tailnet — using Tailscale's identity (your signed-in account) instead of SSH keys.

How it works

When you `ssh julian@apex` from a tailnet-connected device:

1. Tailscale routes the connection through the WireGuard tunnel.
2. The VPS's Tailscale daemon intercepts on port 22 (only over the tailnet).
3. It authenticates using your Tailscale identity (no SSH keys needed).
4. Opens the shell as `julian`.

You can also still SSH using OpenSSH + your normal SSH key over the tailnet — Tailscale doesn't preclude it. The new capability is just that Tailscale identity also works.

Close public port 22

Once Tailscale SSH is working:

```
ssh apex      # confirm Tailscale SSH works
```

Then on the VPS:

```
sudo ufw delete allow OpenSSH
sudo ufw status
```

You should no longer have port 22 open publicly. **Test from a non-tailnet network** (your phone on cellular, for example) — `ssh apex` should fail with timeout. From a tailnet device, it still works.

> **WARNING. TEST BEFORE CLOSING PUBLIC SSH.** If Tailscale SSH isn't working for some reason and you close port 22, you may lock yourself out. Some providers (Hetzner, DigitalOcean) give you a console / rescue mode to recover; others don't. Belt and suspenders: keep your public SSH key on the VPS even when you've disabled the daemon's listening on port 22 publicly — you can re-enable it from the provider console if needed.

The security upgrade

Before Tailscale SSH: - Port 22 publicly open. - Brute-force attempts every minute. - Defense relies on key-only auth + a secure passphrase + fail2ban. - Compromised SSH key = compromised VPS.

After Tailscale SSH + public port 22 closed: - Port 22 not reachable from the public internet. - No brute-force attempts possible. - Defense relies on Tailscale identity (your GitHub/Google/etc. account + Tailscale's auth). - Compromised SSH key alone is useless without also being on the tailnet. - Compromised laptop is bigger blast radius — but that was already true.

It's not "more secure than SSH" in a cryptographic sense; it's "the attack surface is smaller because the attacker isn't on your tailnet."

---

7. Internal services: Caddy on the tailnet

Now: some services on your VPS should be public (your app, your API). Others should be private (admin dashboards, Vaultwarden, Memos, Linkwarden, monitoring). With Tailscale, the private ones can be **only reachable on the tailnet** — not exposed publicly at all.

Approach A: Caddy listens only on the Tailscale interface

```
# In ~/infra/Caddyfile

# Public sites (unchanged)
app.yourdomain.com {
    reverse_proxy app:3000
}

# Internal site bound to Tailscale IP only
vault.yourdomain.com {
    bind 100.y.y.y     # the VPS's tailnet IP

    tls {
        dns cloudflare {env.CLOUDFLARE_API_TOKEN}
    }

    reverse_proxy vaultwarden:80
}
```

The `bind` directive tells Caddy to listen only on the tailnet IP for this site. Requests coming in over the public interface (eth0) on `vault.yourdomain.com` get nothing.

Reload Caddy. Now `https://vault.yourdomain.com` works from a tailnet device; from a public device, it doesn't connect.

> **IMPORTANT. DNS FOR THE INTERNAL SUBDOMAIN MUST BE DNS-ONLY (GRAY CLOUD) IN CLOUDFLARE.** If you proxy it through Cloudflare's edge, CF tries to reach the origin on the public interface — which the *`bind`* directive rejects. Gray-cloud the record; Tailscale takes over from there.

Approach B: Tailscale's HTTPS-on-tailnet

Tailscale can issue HTTPS certs for hostnames inside your tailnet (`vault.tail-c0ffee.ts.net`) automatically. This avoids needing public DNS at all for internal services.

Enable in admin console: DNS → "HTTPS certificates" toggle.

Then on the VPS:

```
sudo tailscale cert vault.tail-c0ffee.ts.net   # generates cert
```

In Caddy:

```
vault.tail-c0ffee.ts.net {
    tls /var/lib/tailscale/certs/vault.tail-c0ffee.ts.net.crt \
        /var/lib/tailscale/certs/vault.tail-c0ffee.ts.net.key

    reverse_proxy vaultwarden:80
}
```

Or skip Caddy entirely and use **Tailscale Serve**:

```
tailscale serve --https=443 --bg http://localhost:8080
```

This makes a local service available over HTTPS on your tailnet — Tailscale handles the cert and routing. Simpler than Caddy for one-off internal services.

Use cases for internal-only services

A short list of things worth self-hosting on your VPS, internal-only:

- **Vaultwarden** — Bitwarden-compatible password manager. Sync between your devices over the tailnet. No third-party trust.
- **Memos / Bytebase** — quick-note services. Like Twitter for yourself.
- **Linkwarden** — read-it-later / bookmark manager.
- **Miniflux / FreshRSS** — RSS reader.
- **Karakeep / Pocket-style** — same vein.
- **Dashy / Homepage** — start page for your other internal services.
- **Adguard Home / Pi-hole** — DNS-level ad blocking; point your devices' DNS at it.
- **n8n** — workflow automation (self-hosted Zapier).
- **Glances / Netdata** — server monitoring.

Each is a single `docker-compose.yml` away. Caddy on the tailnet route, Tailscale identity for access. None of them ever touch the public internet.

---

8. Exit nodes: your VPS as a VPN

A Tailscale **exit node** routes all of a connecting client's traffic through itself. Useful when:

- You're on hostile Wi-Fi (airport, coffee shop, hotel) and don't trust the network.
- You want your traffic to come from your VPS's IP (e.g., for IP-restricted services).
- You're in a country that blocks certain sites, and your VPS isn't.

Enable the VPS as an exit node

On the VPS:

```
sudo tailscale set --advertise-exit-node
```

Then in the admin console, find the VPS device and click "Edit route settings" → enable "Use as exit node."

Use the exit node from your laptop

```
sudo tailscale set --exit-node=apex
```

All your laptop's traffic now goes through the VPS. Verify:

```
curl -s ifconfig.me
# Should return your VPS's public IP, not your local network's
```

Disable:

```
sudo tailscale set --exit-node=
```

Or use the system tray UI to toggle on/off.

> **NOTE. THROUGHPUT IS LIMITED BY THE LINK WITH THE WORST OF YOUR LAPTOP'S CONNECTION AND THE VPS'S.** Latency goes up (an extra hop). For "I'm at a coffee shop and want my banking session encrypted to my VPS, not Comcast's coffee shop AP," this is fine. For streaming 4K video, less so.

What about VPN providers (Mullvad, etc.)

Tailscale partners with Mullvad: you can use Mullvad's exit nodes from your tailnet. Useful when you don't want to expose your VPS's IP, and want a "regular VPN" exit. Costs extra; configure in admin console.

For most personal use, your own VPS exit is fine.

---

9. Subnet routes: bridging existing networks

A subnet route lets one Tailscale node act as a **gateway** to a non-Tailscale network. Useful when:

- You want to reach your home router's `192.168.1.0/24` from your laptop while traveling.
- You have a Raspberry Pi at home running a few services; the Pi runs Tailscale; everything else on the home LAN is reachable via the Pi.
- You're running services on your VPS that listen on Docker's `172.x.x.x` network and want them reachable.

Advertise a route from a node

On the node that has access to the subnet:

```
sudo tailscale set --advertise-routes=192.168.1.0/24
```

Then in admin console: Machines → the node → enable subnet routes.

On clients, the route is now used automatically — `ping 192.168.1.50` works from your laptop, even though your laptop has no direct connection to the home network.

For Docker networks on the VPS:

```
sudo tailscale set --advertise-routes=172.16.0.0/12
```

Now your laptop can directly reach Docker containers on the VPS by their internal IPs (rarely useful, but possible).

---

10. ACLs: who can reach what

By default, every device on your tailnet can reach every other device on every port. For personal use, that's usually fine.

If you ever want finer control:

- Admin console → Access Controls.
- Tailscale ACLs are written in JSON-with-comments (HuJSON).
- You can restrict by tag (e.g., `tag:server` devices reachable only from `tag:laptop` devices), user, port, or any combination.

For most personal setups: don't bother. The default-allow within a small tailnet is fine. Revisit if you ever invite collaborators (Tailscale supports up to 3 users on free).

Tags

A useful pattern: tag your devices.

```
{
    "tagOwners": {
        "tag:server": ["joshuawjulian@gmail.com"],
        "tag:laptop": ["joshuawjulian@gmail.com"]
    },
    "acls": [
        {
            "action": "accept",
            "src": ["tag:laptop"],
            "dst": ["tag:server:22,80,443"]
        }
    ]
}
```

Then assign tags to devices via `tailscale up --advertise-tags=tag:server` or in the admin console.

Tags make ACLs readable as the tailnet grows. Skip until you have 5+ devices and start wanting differentiation.

---

11. Funnel: selectively expose to the public

Tailscale **Funnel** is the inverse of everything we've been doing: it makes a service on your tailnet **reachable from the public internet** through Tailscale's edge — without opening any ports on your VPS, without DNS configuration, without a separate reverse proxy.

Use case: you want to share one service publicly (e.g., a demo of a project, a temporary webhook receiver) but don't want to do the whole Caddy + DNS setup.

Enable Funnel

Admin console → DNS → "Funnel" → enable.

On the device hosting the service:

```
tailscale funnel 80
# or
tailscale serve --bg --https=443 --funnel http://localhost:8080
```

This advertises the service publicly at `https://<machine>.tail-c0ffee.ts.net` (Tailscale's domain, with your Funnel-enabled hostname). The request reaches Tailscale's edge, gets forwarded through the tailnet to your service.

Disable:

```
tailscale funnel off
```

> **NOTE. FUNNEL IS BEST FOR TEMPORARY PUBLIC EXPOSURE.** Hostname is `<machine>.tail-c0ffee.ts.net` — not a custom domain. For permanent public services with branding, use Caddy + your own domain (per the domain-caddy-https guide). For "I just need to demo this to someone for an hour," Funnel is unbeatable.

---

12. Removing a device

Lost a laptop, decommissioning a VPS, or want to deauth a device:

- **Admin console** → Machines → click the device → "Delete." The device is removed; its tailnet IP is freed; its keys are revoked.
- Or from another tailnet device: `tailscale logout` while logged in to the device (forces local re-auth).

If you can't reach the device (lost laptop), the admin console deletion is the path. The device's local Tailscale daemon will fail to reconnect.

---

13. Troubleshooting

`tailscale up` opens a browser to login but URL never works

You're on a server with no browser. Open the URL manually in a browser on another device — it's authenticating against your Tailscale account, not the specific device. Approval flows through the admin console.

`tailscale status` shows all peers as "DERP relay"

Your connection isn't punching through to peers directly. Possible causes:

- Strict NAT / symmetric NAT on one side.
- UDP blocked on a corporate firewall.

Tailscale falls back to DERP relay servers (Tailscale's own infrastructure) in this case. Performance is fine for most uses (DERPs are HTTP/2 over TCP, decent throughput) but latency-sensitive workloads notice. Try `tailscale up --advertise-exit-node` on a node to see if NAT traversal can be coaxed.

Tailscale SSH doesn't work

```
ssh -v apex
# debug1: Authentication succeeded (publickey).
# debug1: Authentication succeeded (publickey).
# ...
# Permission denied
```

Verify on the VPS:

```
sudo tailscale status   # shows the daemon is healthy
sudo systemctl status tailscaled
sudo journalctl -u tailscaled --since "10 minutes ago"
```

Common causes: - `tailscale up --ssh` wasn't run. Re-run with `--ssh`. - ACLs are blocking. Check admin console. - The user account on the server doesn't match your Tailscale identity. By default Tailscale SSH lets you log in as any user that allows you; explicit ACLs may restrict.

"DNS not working" inside Docker containers after enabling MagicDNS

Tailscale's MagicDNS sets the system resolver to point at Tailscale's resolver. Docker containers inherit DNS from the host by default, but the host now resolves via `100.100.100.100` — which works for tailnet names but may not work for public DNS for some setups.

Fix in `daemon.json` or docker-compose:

```
services:
  app:
    dns:
      - 1.1.1.1
      - 100.100.100.100   # tailnet
```

Or accept that containers use only public DNS and don't need to resolve tailnet hostnames.

Tailscale eats my battery on mobile

On Android, the Tailscale VPN is constantly maintaining tunnels. It uses some battery. Tweak: in the app, disable "Always on VPN" and enable on demand.

On iOS, less of an issue — system VPN is more aggressive about sleep.

---

14. Alternatives considered

Mesh VPNs

- **Tailscale** (this guide) — managed, free for personal, just works.
- **Headscale** — open-source, self-hostable coordination server compatible with the Tailscale clients. Reconsider when you want to fully self-host (no Tailscale Inc. trust dependency).
- **NetBird** — open-source competitor. Smaller community as of 2026.
- **ZeroTier** — older mesh VPN. Works; less polished than Tailscale's modern era.

Plain WireGuard

- **Set up your own WireGuard server** — feasible but you handle key exchange, NAT traversal (most homes are behind double-NAT now), peer config, IP allocation, hostname conventions. Tailscale handles all of that.
- Reconsider when: you want zero outside-vendor dependency and you're comfortable doing all the operational work. For personal use this almost never pays off.

Traditional VPNs

- **OpenVPN** — works but bloated and slow compared to WireGuard.
- **Strongswan / IPsec** — corporate-grade; way too much.
- **Commercial VPN providers (Mullvad, ProtonVPN, etc.)** — for privacy-out-to-the-public-internet, not for private mesh between your devices. Different problem.

Exposing specific services without a tailnet

- **Cloudflare Tunnel** — public-internet ingress without opening firewall ports; runs an outbound tunnel from your VPS to Cloudflare's edge. Reconsider for services that must be public anyway. For private-to-you services, Tailscale beats it (no Cloudflare in the path).
- **ngrok** — like Funnel but commercial, more features. Reconsider for temporary public exposure if you want the extra features. For permanent personal use, Tailscale Funnel is free.

For SSH specifically

- **OpenSSH + fail2ban + non-standard port** — works; lots of ongoing maintenance, still public-facing.
- **SSH certificates from a CA** — strong, but organizational overhead.
- **Tailscale SSH** — easiest path to "port 22 is not publicly reachable at all."

---

15. Quick reference

Setup (per device)

```
# Linux / WSL
curl -fsSL https://tailscale.com/install.sh | sh
sudo tailscale up --ssh    # add --ssh on servers; omit on laptops

# macOS
brew install --cask tailscale
# then sign in via the menu bar app

# Windows: download installer from tailscale.com
# iOS / Android: App Store / Play Store
```

Status

```
tailscale status            # all peers + IPs
tailscale ip -4             # this device's IPv4 in the tailnet
tailscale ip -6             # IPv6
tailscale netcheck          # connection quality + NAT type
```

Common operations

```
# Use VPS as exit node
sudo tailscale set --exit-node=apex
sudo tailscale set --exit-node=          # disable

# Advertise this node as an exit node
sudo tailscale set --advertise-exit-node

# Advertise subnet routes
sudo tailscale set --advertise-routes=192.168.1.0/24

# Tags
sudo tailscale set --advertise-tags=tag:server

# Logout (forces re-auth)
sudo tailscale logout
```

Funnel

```
tailscale funnel 8080                                  # expose port 8080 publicly
tailscale serve --bg --https=443 --funnel http://localhost:8080
tailscale funnel off                                   # stop
tailscale funnel status                                # what's exposed
```

Internal HTTPS via Tailscale-issued certs

```
sudo tailscale cert <hostname>.tail-c0ffee.ts.net
# certs in /var/lib/tailscale/certs/
```

Closing public SSH after Tailscale SSH works

```
# On VPS, after testing Tailscale SSH:
sudo ufw delete allow OpenSSH
sudo ufw status
# Verify from a non-tailnet device that public SSH no longer connects
```

Where the admin console lives

https://login.tailscale.com/admin — devices, DNS, ACLs, Funnel toggles, auth keys.